

Themison Platform - Detailed Development Accomplishments

This document provides a highly granular summary of the work completed on the **Themison Clinical Trial Platform**, explaining **what** was done, **why** it was done, and **why it is important** (its business/operational impact).

1. Relational Database Consolidation (PostgreSQL Migration)

- **What was done:**
 - Completely removed the secondary MySQL database engine configuration and frontend/backend dependencies.
 - Consolidated all tables, schemas, and configurations into a single, unified PostgreSQL database.
 - Created and applied the Alembic database migration blueprint (`0b36ae7e7c90_add_task_manager_fields_and_dependencies.py`) to upgrade live tables.
 - **Why (The Reason):** Maintaining dual database environments (MySQL and PostgreSQL) simultaneously caused queries to fail, schemas to diverge, and introduced high synchronization overhead.
 - **Why it is Important (Impact):** Simplifies platform infrastructure, eliminates database synchronization lags, guarantees data integrity, and enables the use of advanced Postgres-specific features (such as large vector embeddings and HNSW indexes for semantic AI RAG search) in a single database.
-

2. Interactive Task Manager & Dependency Engine

- **What was done:**
 - **Live Database Connection:** Connected the 4,000+ line interactive Kanban/Gantt task board interface (`Tasks.tsx`) to the live PostgreSQL database via tRPC proxy endpoints, replacing all in-memory mock data.
 - **Schema Fields Expansion:** Added crucial fields to the tasks schema:
 - `phase_id`: Links tasks to trial phases/visits.
 - `assigned_role`: Supports role-based task delegation (PI, CRC, Nurse, Monitor).
 - `blocked_reason / blocked_since`: Supports structured JSON blocker metadata.
 - `order_in_phase`: Stores custom drag-and-drop orders.
 - `suggested_date / suggested_assignee`: Added fields for AI-driven task scheduling.
 - **Task Reordering Logic:** Programmed state updates, splicing, and lazy-loading logic in drop handlers to enable drag-and-drop task reordering.
 - **Advanced Drag-and-Drop Library:** Replaced raw HTML5 Drag APIs with `@dnd-kit/core` and `@dnd-kit/sortable` wrappers, useSortable hooks, and SortableContext wrappers for smooth drag interactions.
 - **Drag-and-Drop Visual States:** Added drag-over highlights, drag-start opacity, cursor-move changes, and visual indicators.
 - **Interactive Task Dependencies:** Created the `task_dependencies` database table to track finishes-to-starts and cross-phase relationships, replacing static stubs.
 - **Edit Integrations:** Added interactive hover-state pencil edit buttons on task cards and wired them to edit callbacks.
 - **Page Background Alignments:** Adjusted styles to match the global trial workspace theme and configured task sorting by descending priority.
 - **Why (The Reason):** Task board data was previously lost on page refresh, and drag-and-drop operations were non-functional or jittery.
 - **Why it is Important (Impact):** Trial coordinators can now orchestrate clinical operations in real-time, link dependencies, reorder phases, and log task blockers securely.
-

3. Clinical Study Setup Wizard (UI/UX Transformation)

- **What was done:**
 - **Stepped Interface Flow:** Redesigned the Wizard UI into a structured 3-step sequence:
 1. *Upload Documents:* Protocol upload checklist.
 2. *Review Coverage:* AI coverage completeness progress bars.
 3. *Generate Plan:* Operationalized task preview list.
 - **Multi-Document Checklist:** Implemented support for Required (Protocol with page count indicators) and Recommended auxiliary documents (Monitoring Plan, Safety Reporting Manual, EDC/CRF Completion Guide).
 - **Dynamic Custom Document Types:** Added a [Add Other Document Type] custom builder allowing users to specify and upload custom files.
 - **Bidirectional Document Syncing:** Linked Wizard upload buttons to the Document Hub upload API, resolving categorization gaps so files appear identically in both places.
 - **Scrolling & Spacing Fixes:** Fixed the Wizard header and navigation footer at the margins to prevent content cutoff, shifting scroll operations strictly to the middle step content.
 - **Polished Step Indicators:** Added SVG arrow indicators, light green backgrounds (#edfcf2), and customized border icons (#62D686) for completed steps.
 - **Aesthetic Alignment:** Matched container margins to trial tab bars and progress indicators to the workspace brand colors.
 - **Why (The Reason):** The original interface looked like a colorful consumer app, lacked scrolling control, and was restricted to uploading one file.
 - **Why it is Important (Impact):** Gives clinical coordinators a high-trust, professional interface to manage all trial startup documents and estimate task coverage before operational plans are generated.
-

4. Document Hub UI Redesigns & Category Badges

- **What was done:**
 - **Inline Layout Adjustments:** Moved the "Documents" header and document count inside the white page body container on a single line with the upload button.
 - **Padding & Alignments:** Removed container left padding and aligned the margins with the tab bar.
 - **Editable Categories:** Converted document categories into editable inline dropdown menus, saving new custom classifications back to the database.
 - **Automatic Processing:** Programmed automatic document processing triggers immediately on upload, adding delete handlers and live status indicators (uploaded, processing, indexed, error).
 - **Why (The Reason):** The document listing layout was misaligned with the dashboard grid and required manual clicking to trigger vector indexing.
 - **Why it is Important (Impact):** Provides a seamless document management experience where uploaded files are automatically parsed, categorized, and indexed for immediate AI search.
-

5. Themison AI Chat Assistant (Figma Pixel-Perfect Implementation)

- **What was done:**
 - **Two-State Layout:** Designed a ChatGPT-style empty state (with custom prompt cards and a "Show more" button) that transitions into a full-height chat window once a message is typed.
 - **Layout Spacing Improvements:** Shifted container padding (pt -16 to pt -8) and margins to eliminate white space gaps and overlaps.
 - **Floating PDF Preview Pane:** Built a responsive sliding side panel that adjusts chat window widths, featuring rounded corners (rounded-t l -1g), negative margin offsets (-m l -2), and header close/expand buttons.

- **UI Input Bar Tweaks:** Changed textarea backgrounds to white, removed borders, and custom-colored control buttons (paperclip, Auto, microphone) to #F3F3F5 with custom dark-gray hover highlights.
 - **Microphone Input:** Added a voice input microphone button.
 - **Send Arrow Icon:** Changed the default send paper plane icon to an ArrowUp SVG icon with customized dark hover actions.
 - **Markdown Response Formatting:** Added react-markdown and react-syntax-highlighter to render code formatting and tables.
 - **Lead Paragraph Emphasis:** Configured first paragraphs of AI responses to render in larger, bold text.
 - **Collapsible Thoughts UI:** Created an expandable section showing the AI's internal reasoning process above the final answer.
 - **Text Overflow Wrap:** Added break-words styling to prevent long text blocks from overflowing.
 - **Why (The Reason):** The previous layout was a static mockup with non-functional controls, missing markdown rendering, and layout offsets that overlapped top navigation tabs.
 - **Why it is Important (Impact):** Offers a highly polished chat interface where researchers can ask questions about protocols, read clean markdown responses, collapse reasoning steps, and view cited documents in a side-by-side layout.
-

6. Advanced AI RAG Pipeline & Vector Search

- **What was done:**
 - **OpenAI Assistant Migration:** Migrated the backend RAG architecture from Google File Search to the OpenAI Assistants API.
 - **Vector Store Management:** Created tables and backend helpers to manage isolated Vector Stores per trial.
 - **Document-Level Filtering:** Implemented a multi-select document checklist inside the AI "Source" modal, updating backend chat routes to query documents by ID.
 - **Search Scope Indicator:** Created an input-width matched search scope status bar (bg-white, rounded-2xl, border) indicating if the AI is searching "All Documents" or a specific subset, with text truncation and a non-shrinking "Clear filter" button.
 - **PDF Extraction & Highlight Fallbacks:** Configured pdf-parse text extractions with Node 20+ DOMMatrix polyfills. Built a Python PyMuPDF (fitz) fallback script to highlight sentences locally when external citation services are offline.
 - **URL Correction & Fail-safe Cache:** Added self-healing URL converters for localhost and fail-safe Redis cache configurations.
 - **Why (The Reason):** The AI assistant could not read actual document content previously, lacked trial isolation, and coordinate highlight coordinates broke during local/offline testing.
 - **Why it is Important (Impact):** Guarantees accurate, secure, and auditable answers grounded in trial documents, showing exact text citation highlights even when external systems are offline.
-

7. Trial Metadata, Sidebar & Routing Synchronization

- **What was done:**
 - **Editable Trial Fields:** Created a reusable EditableField component with hover-to-edit pencil icons. Added trRPC routes to edit Trial Title, Protocol Number, Description, Phase, Sponsor, Location, and Start/End dates.
 - **Live Sidebar Synchronization:** Replaced hardcoded trial cards in the Sidebar navigation and Workspace metrics with a live trpc.trials.list database query. Configured cache invalidation to update the sidebar in real-time when details are edited.
 - **Slug & Breadcrumb Synchronization:** Updated TopNav breadcrumbs to fetch trial details from the database and resolved numeric vs. alphanumeric slug routing errors (preventing NaN slug page crashes).
 - **Why (The Reason):** The platform previously used hardcoded mock data for trial stats, leading to database updates not appearing in the sidebar or headers.
 - **Why it is Important (Impact):** Unifies all trial details across pages, breadcrumbs, sidebar menus, and headers under a single database source of truth.
-

8. Development Verification Status

- **Unit & Integration Tests:** Successful Vitest test executions on document uploads, trials updating mutations, and RAG retrieval.
- **Development Seeding:** Successfully implemented `seed-wizard-demo.mjs` and `seed-trials.mjs` to populate PostgreSQL tables.